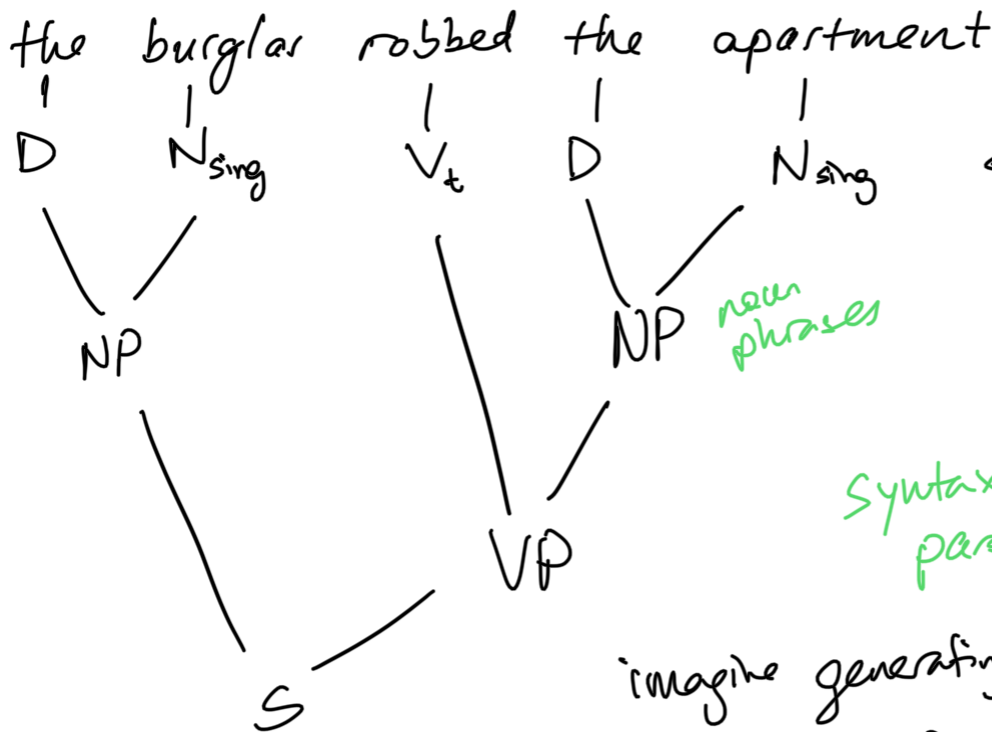


Context-Free Grammars

In linguistics,
this is called
SYNTAX



← POS tags
(HMM could
model there)

Syntax tree /
parse tree

imagine generating this with rules

S (start symbol) → NP VP

NP → D N_{sing}

NP → N_{pl}

VP → V_t NP

NP → D N_{pl}

V_t → robbed

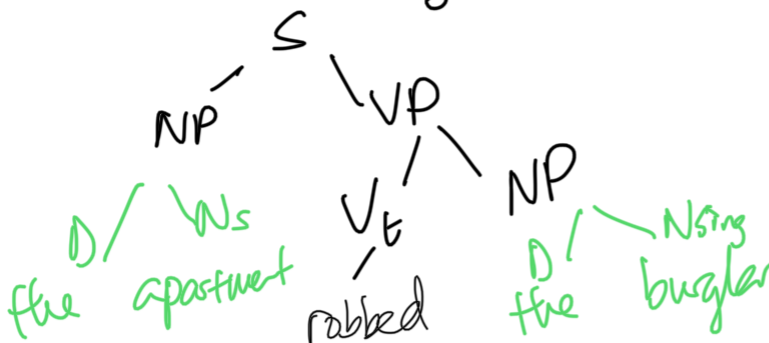
D → the

N_{sing} → burglar

N_{sing} → apartment

the/a cat
the/∅ cats

even w/ this super limited
rule set, could generate 4 sentences



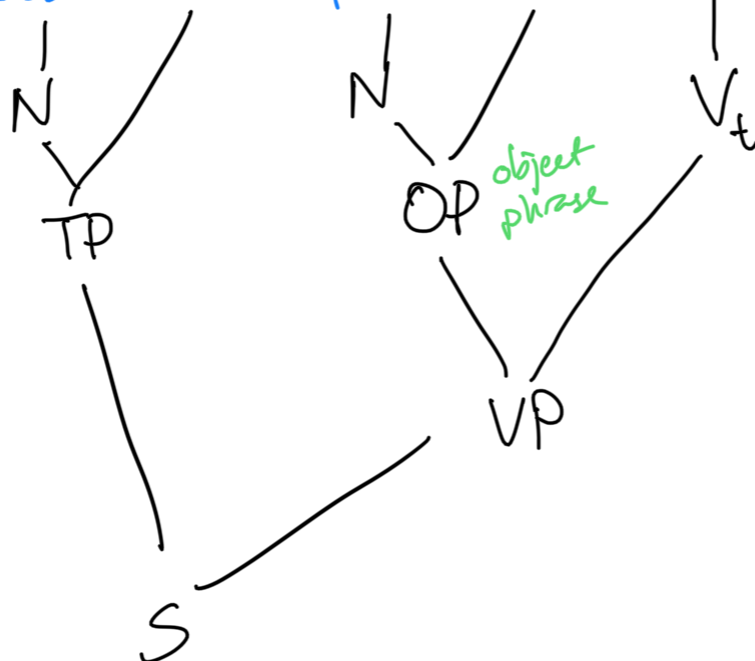
etc.

Different in every language

Japanese

泥棒はアパートを略奪した

transcription dorobō wa apāto wo ryakudatsushita
gloss robber TOPIC apartment OBJ robbed



$S \rightarrow TP VP$
 $TP \rightarrow N \text{ TOPIC}$
 $VP \rightarrow OP V_t$

Def Context-Free Grammar (CFG) is

- N set of non-terminals
- Σ set of terminals (words)
- R is rule set, composed of rules:

$X \rightarrow Y_1 Y_2 \dots Y_n \quad n \geq 0$

$X \in N$

$Y_i \in N \cup \Sigma$

- S start symbol

NP VP TP
D
like V in previous models

all non-terms
 $NP \rightarrow D N$
JA $TP \rightarrow N \text{ wa}$
 $\in N \in \Sigma$

Formal idea of "deriving sentences"

Def Left-most derivation is a sequence $S_1, \dots, S_n$

$S_1 = S$ (start symbol)

$S_n \in \Sigma^*$ terminals/words only

S_n called "yield" of derivation

$S_i \in (N \cup \Sigma)^*$ (mixed)

structure: for $i = 2, \dots, n$

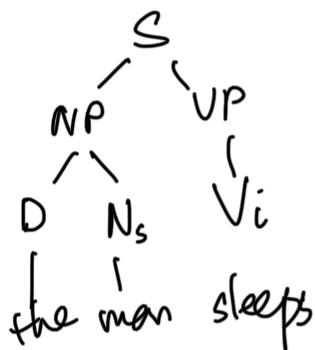
$S_i = S_{i-1}$ but the leftmost non-terminal X

is replaced by $Y_1 \dots Y_n \in (N \cup \Sigma)^*$ where

$X \rightarrow Y_1 \dots Y_n \in R$

ex the man sleeps

left-most der



↔
bijection

S
 $NP \quad VP$
 $D \quad N \quad VP$
 the $N \quad VP$
 the man VP
 the man Vi
 the man sleeps

sleeps is non-transitive
(no object)

$VP \rightarrow V_i \rightarrow \text{add to } N$
 $V_i \rightarrow \text{sleeps} \rightarrow \text{add to } \Sigma$
 $N_c \rightarrow \text{man} \in \Sigma$

left-most der is depth-first construction of syntax tree

CFG specifies set of its left-most derivations

say $s \in \Sigma^*$ is in the language of the CFG if

there is at least one left-most der. whose yield is s

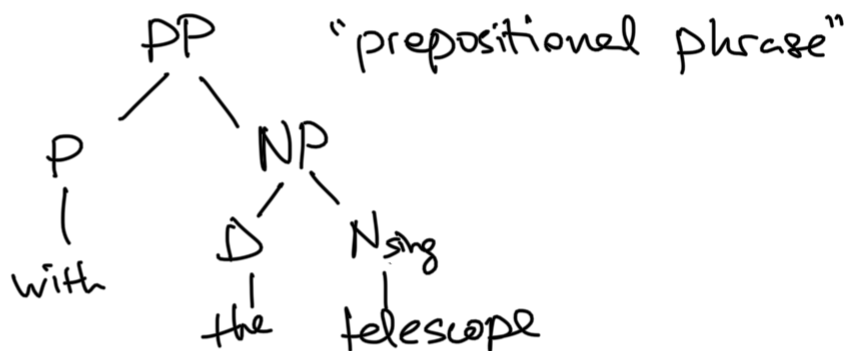
... not in case of our simple CFG

eg the sleepy man

At least one?

Could be many...

the man saw the dog with the telescope.



where can PP go?

I eat with a fork

delicious
fish
slowly

the man with a fork rocks

VP ? wrong

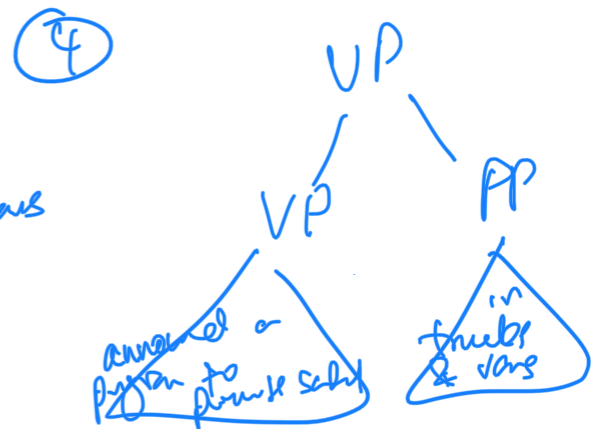
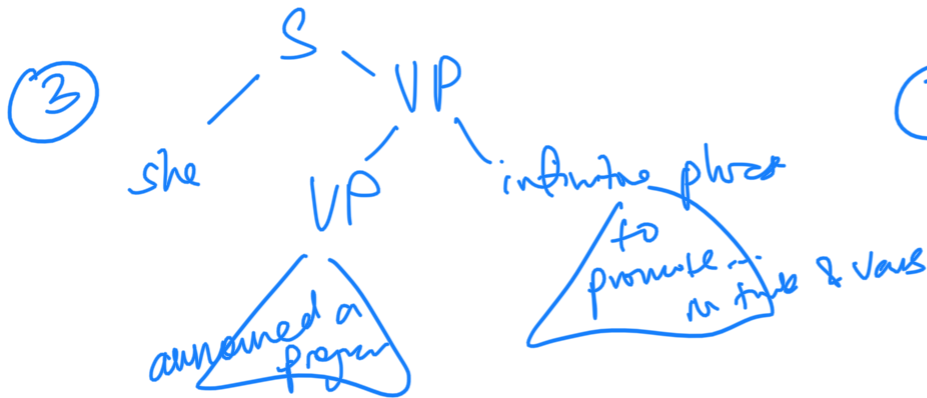
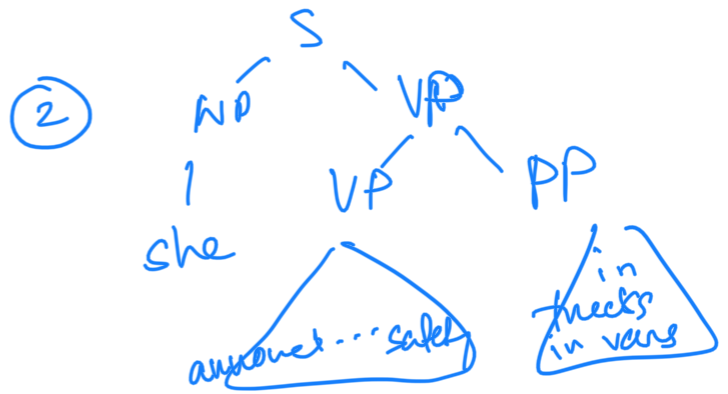
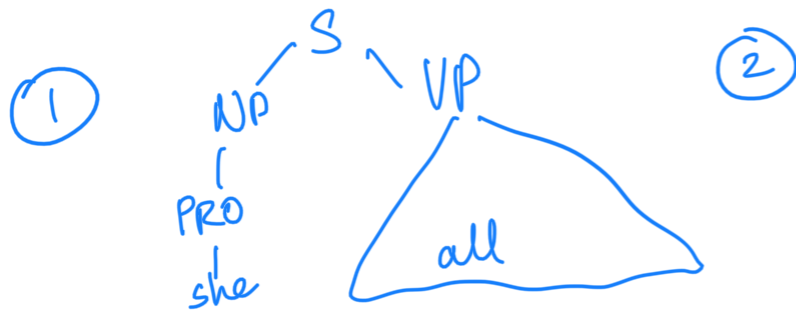
VP ✓

NP
NP PP

worse examples

she announced a program to promote safety
in trucks and vans

at least 14 parses / derivations



Ambiguity: when S has > 1 l.m.d.

English Grammars: deeper!

SO FAR

Nouns: singular N_{sing}, NN
 plural N_{pl} NNS
 proper NNP

Just

Determiners: D, DT

the, a, some, every

Adjectives: ADJ / JJ

Prepositions: PROP, P, IN

with, on, of, in

— ← called N-bar

he knows that the man sleeps

that the mechanic saw the car sleeps

new rules & verb type

V_{comp}

$VP \rightarrow V_{comp} \bar{S}$

$V_{comp} \rightarrow$ said
reported
knows

$\bar{S} \rightarrow COMP S$

$V_{trans-comp}$

$VP \rightarrow V_{trans-comp} NP \bar{S}$

$V_{ditrans-comp}$

$VP \rightarrow V_{ditrans-comp} NP NP \bar{S}$ eg
let

the man let his friend \$50 that the woman has a car

CC = coordinator (simpler than COMP)

eg and, or, but

I love chocolate and you love wine
S S

$S \rightarrow S CC S$

$\bar{S} \rightarrow \bar{S} CC \bar{S}$

he knows that I am the king
and that you love wine
S

$NP \rightarrow NP CC NP$

$VP \rightarrow VP CC VP$

Back to CFG theory.

$G = CFG$

let $T_G =$ all l.m.d (free) under G

... and $f(t) = s$ is yield of $t \in T_G$

$$T_G(s) = \{ t \in T_G \text{ s.t. } \text{yield}(t) = s \}$$

s is ambiguous if $|T_G(s)| > 1$

s is in the lang of G $|T_G(s)| > 0$ also say s is "grammatical in G "

Want to make machine learning
i.e. add probability to CFG

Want: $\forall t \in T_G, p(t) \geq 0$ and $\sum_{t \in T_G} p(t) = 1$

i.e. prob dist on trees / l.m. derivs

why?

- gives language-model (if you look at just yields)

i.e. like HMM ... LM + extra structure

HMMs: hidden tag seq

PCFGs: syntactic tree / l.m. deriv

i.e. HMM / PCFGs both joint models on sentences + hidden structure

Also, given $s \in \Sigma^*$, can get its syntactic structure
look at trees $t \in T_G(s)$ (i.e. trees yielding s)

$$t^* = \arg \max_{t \in T_G(s)} p(t)$$

① How to define valid p ?

② Given that, how to solve arg max problem?

Def Probabilistic Context Free Grammar (PCFG) is

- CFG $G = (N, \Sigma, R, S)$

- parameter $q: R \rightarrow [0, 1]$

For rule $X \rightarrow \beta \in R$

$q(X \rightarrow \beta)$ is conditional prob of choosing rule $X \rightarrow \beta$ in a l.m.d.

given that the non-term being expanded is X .

$\forall X \in N$

$$\sum_{(X \rightarrow \beta) \in R} q(X \rightarrow \beta) = 1$$

Given $t \in T_G$ composed of rules $X_1 \rightarrow \beta_1, X_2 \rightarrow \beta_2,$

$\dots, X_n \rightarrow \beta_n$

$$p(t) = \prod_{i=1}^n q(X_i \rightarrow \beta_i)$$

when does this give legit dist over Σ^* (yields)....?

Assumption behind all this (modeling prob dist of possible rules expanding each non-term)

Sentences / parse trees generated stochastically via following process:

$S_i := S, i := 1$

while (S_i contains non-term)

$X := \text{left-most non-term in } S_i$

.....

choose $X \rightarrow \beta$ by distribution $q(X \rightarrow \beta)$

$S_{i+1} = S_i$ replacing X by β

$i++$

Supervised learning for PCFGs :

Given parse tree (with their yields / sentences)

$C = \{t_i\}_{i=1}^m$, $\forall t_i$ have same start symbol S

also $C \rightarrow$ outputs N (non-terminals in $t_1, \dots, t_m$)
 Σ (words in $t_1, \dots, t_m$)

and R and q

Spoiler MLE estimates (parameters that maximize $\text{prob of } C$)

are $R =$ all rules seen ≥ 1 in C

$$q_{\text{ML}}(X \rightarrow \beta) = \frac{\text{Count}(X \rightarrow \beta)}{\text{Count}(X)}$$



found 105 times in C

VP found 1000 times (895 times



$$q_{\text{ML}}(VP \rightarrow V_t NP) = \frac{105}{1000}$$

Since all VPs expanded by some rule, trivially q_{ML} gives valid prob. dist over the rules expanding VP

Q2 Decode / parse

Given PCFG $\mathcal{G}$ with q , sentence s

Want: $\arg \max_{t \in \mathcal{T}_{\mathcal{G}}(s)} q(t)$

Next time

Clever DP algo

Even brute force not obvious